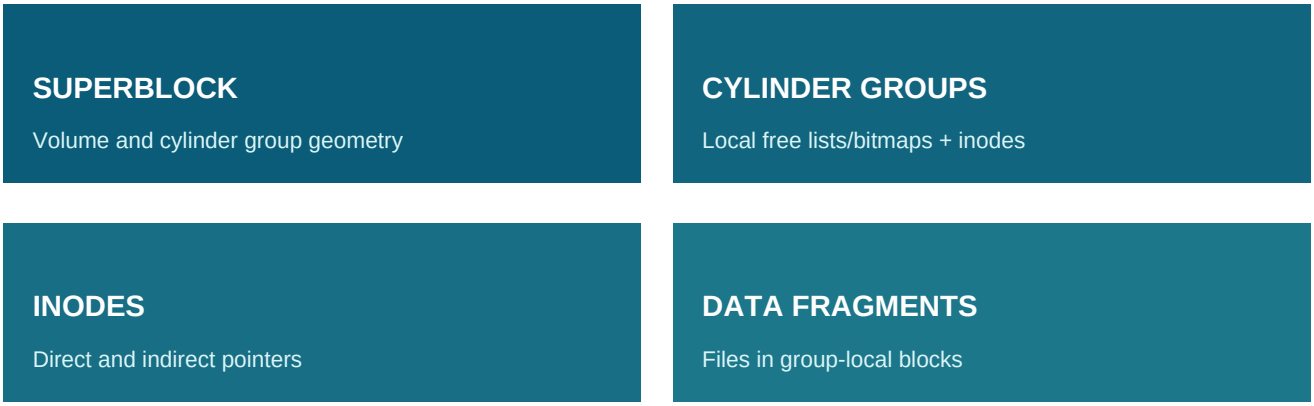
UFS1 - BSD Fast File System heritage

HISTORY / ORIGINAL DESIGN / DISK FORMAT

The classic Berkeley Unix Fast File System layout: cylinder groups, inodes, allocation bitmaps and carefully chosen block placement.

This is a technical reference to the upstream filesystem's historical design, not an installer manual. The format's on-media semantics are distinct from the independent InfiltratorOS module, its APIs and any Linux kernel adapter.

CONCEPTUAL MEDIA MAP



CONTENTS

1. Historical chronology
2. Original on-disk architecture
3. Allocation and directories
4. Integrity and recovery
5. Design rationale and limitations
6. InfiltratorOS integration boundary

UFS1 - BSD Fast File System heritage

HISTORICAL CHRONOLOGY

The Berkeley Fast File System was designed by Marshall Kirk McKusick and colleagues in the early 1980s to overcome severe fragmentation and performance limits of earlier Unix filesystems on magnetic disks. Their 1984 paper, A Fast File System for UNIX, documents the engineering rationale. UFS1 represents the classic on-disk inode and cylinder-group family used by BSD-derived systems.

UFS1 grew through BSD and FreeBSD evolution, with soft updates later providing dependency-ordered metadata changes and optional journalling mechanisms appearing in some operating systems. UFS2 introduced a new format for larger volumes and more metadata capacity. UFS1 and UFS2 share conceptual architecture but not interchangeable on-disk inode structures.

UFS1 - BSD Fast File System heritage

ORIGINAL ON-DISK ARCHITECTURE

A UFS1 volume has a superblock with parameters defining block sizes, fragment sizes and cylinder group geometry. Each cylinder group includes local bookkeeping: allocation information, inode areas, replicated critical metadata and data blocks. The original design explicitly places related inodes and blocks close to one another to reduce seeks on rotating media.

Inodes store mode, ownership, timestamps, link count, size and a direct/indirect block addressing scheme. Large files require indirect pointer blocks. Fragments allow the final portion of a small file to consume less than a full block. The disk’s real format variant, byte order and filesystem parameters determine exact field offsets; an ext2 inode cannot be substituted because both are Unix-like.

UFS1 - BSD Fast File System heritage

ALLOCATION AND DIRECTORIES

Cylinder groups are the primary allocation unit, with policies spreading directories and clustering the data of individual files. A free-space bitmap and summary information help the allocator find blocks and fragments. Full block allocation favours performance, while fragment allocation limits tail waste. Contemporary media no longer have the original seek assumptions, but format-level structures remain.

Directories contain name-to-inode records. Hard links reference the same inode and share data; file removal changes link counts and allocator state. Directories must be scanned with explicit record-length and name-length bounds. The root inode and directory hierarchy must be consistent with group allocation metadata.

UFS1 - BSD Fast File System heritage

INTEGRITY AND RECOVERY

Classic UFS1 did not start with a modern metadata journal. Its reliability depended on write ordering, synchronous updates and offline fsck. Soft updates introduced dependency tracking to order metadata writes safely with fewer synchronous operations; it was an implementation improvement and not a universal property of every UFS1 image.

Recovery checks cylinder group counts, allocated blocks, inode references, directory connectivity and link counts. Soft-updates state and other platform-specific recovery extensions require separate interpretation. A filesystem checker cannot regard replicated superblocks as proof that the last directory update completed.

UFS1 - BSD Fast File System heritage

DESIGN RATIONALE AND LIMITATIONS

The principal breakthrough was allocation locality, larger transfer units and block fragments rather than a novel journal. The implementation balanced sequential throughput and space efficiency by selecting block and fragment sizes, spreading inodes and localising access. Original policies assume properties of rotating magnetic media.

UFS1 inode, timestamp and capacity widths constrained later deployments. UFS2 was designed because incremental expansion of UFS1 was not sufficient for multi-terabyte systems. Current FreeBSD support may differ from historical BSD, and physical layout details vary by implementation version.

UFS1 - BSD Fast File System heritage

INFILTRATOROS INTEGRATION BOUNDARY

UFS1.so is a distinct on-media format module. Sharing an internal API with UFS2.so is legitimate, but superblock, inode and extent/addressing checks must remain format-specific. A runtime probe must distinguish variants before enabling any write operations.

Original BSD papers and FreeBSD implementations are the technical sources. Software package installation and optional kernel adapters are separate from UFS1’s historical cylinder-group format.

UFS1 - BSD Fast File System heritage

SOURCES AND PRIMARY REFERENCES

The documents below are the historical and engineering reference points used for this guide. Where an on-media field or feature flag is ambiguous, consult the original format documentation and its version-specific implementation notes before interpreting or changing media.

1. McKusick et al., A Fast File System for UNIX (1984 original paper)

https://web.eecs.umich.edu/~prabal/teaching/eecs582-w11/readings/mckusick84ffs.pdf

2. USENIX, Soft Updates technique (1999)

https://www.usenix.org/conference/1999-usenix-annual-technical-conference/soft-updates-technique-eliminating-most

The InfiltratorOS shared-object module, optional kernel adapter, native builder and release-specific tests are a separate implementation. A successful document build or package installation does not qualify physical writes, recovery, kernel mounting or boot.

UFS1 / engineering detail

DISK READ PATH / ENGINEERING NOTES

1. Find and validate the UFS1 superblock, then read block size, fragment size, cylinder-group geometry, inode counts and allocation summaries.

2. Translate cylinder-group-relative offsets into filesystem addresses using the format's superblock parameters. Cylinder groups organise metadata and file locality.

3. Locate an inode through its cylinder group and inode-table position. UFS1 inode fields and block pointers are not the same as UFS2's widened structures.

4. Follow direct and multiple levels of indirect block pointers to reconstruct file order. A file may use fragments for its tail rather than whole filesystem blocks.

5. Decode directory records with record lengths, inode numbers and name lengths; check forward progress and inode bounds throughout every directory block.

6. Interpret hard links as multiple directory entries referencing one inode and maintain inode link counts separately from allocation ownership.

7. Read free-block, fragment and inode usage maps in each cylinder group rather than relying solely on the volume-wide free-space summary.

8. Distinguish filesystem format from update policy: traditional synchronous updates, soft updates and later journaling facilities do not all imply identical disk layouts.

UFS1 / engineering detail

VALIDATION, RECOVERY AND COMPATIBILITY

1. Verify superblock and cylinder-group magic, ranges, offsets and allocation counts before using their pointers for writable media access.

2. Check direct/indirect pointer traversal for duplicate ownership, cycles and references outside permitted block or fragment ranges.

3. Cross-check cylinder-group free maps with inode allocations and referenced data fragments. Aggregate free-space totals can be stale or inconsistent.

4. Respect the format’s disk byte order and field sizes; a host-native structure cast is unsafe for images written by a different architecture.

5. Validate variable-size directory records and inode link counts before determining whether a file has been orphaned.

6. Soft updates improve dependency ordering but are not a copy-on-write transaction journal. Interrupted operations still require appropriate checking.

7. Treat alternate superblocks as evidence to corroborate geometry, not as guaranteed current copies of every other metadata structure.

8. Do not write a UFS2 volume using UFS1 inode offsets, even if both share cylinder groups and familiar BSD filesystem terminology.